BANKING MANAGEMENT SYSTEM

Software Requirements Specification

Functional and non-functional requirements for the C, TCP, file-backed implementation

Implementation-specific documentation

Important: Features and limitations are documented from the project source files and README. Tests are not claimed as independently executed for this documentation package.

1. Purpose

This specification captures the observable behavior described in the project for a terminal-based banking application with four roles. It is based on README and source-level command definitions; it does not invent requirements for a database-backed or web implementation.

2. Product scope and context

The product consists of a CLI client, a TCP server, a file layer included by server.c, five fixed-width files under data/, a bank.log audit trail, and a stress-test executable. The server listens on TCP port 8080 and each request is served on a separate connection/process.

3. User classes and role IDs

ID	Role	Principal duties
1	Customer	Own account transactions, passbook, loan applications, feedback
2	Bank Employee	Customer support and assigned loan processing
3	Manager	Activation/deactivation, loan assignment, feedback review
4	Administrator	Employee creation and user/role maintenance

4. Functional requirements

ID	Requirement	Description
FR-01	Login	Allow a user to log in with username/password and selected role; reject invalid credentials, role mismatch, inactive users, and duplicate simultaneous sessions.
FR-02	Customer registration	Allow customer self-registration; create user and account records and return generated IDs.
FR-03	View balance	Allow a customer to view the balance associated with their account.
FR-04	Deposit	Accept only valid positive amounts; update the balance under a record lock and append a ledger entry.
FR-05	Withdraw	Reject invalid amounts or insufficient funds; update balance under a record lock and append a ledger entry.
FR-06	Transfer	Validate recipient and amount; lock both accounts in a deterministic order; update balances and add ledger entries.
FR-07	Transaction history	Allow a customer to view transaction history for their account.
FR-08	Loan application	Allow a customer to submit a positive loan request; initial status is PENDING; customer can view own applications.
FR-09	Feedback	Allow customer feedback submission; manager can review stored feedback.
FR-10	Employee customer support	Allow employees to create customers with an opening balance, reset customer passwords, and view a customer passbook.
FR-11	Assigned loan processing	Allow employees to view loans assigned to them and approve/reject only a loan assigned to their own user ID and not already processed.
FR-12	Manager account control	Allow managers to activate/deactivate customer users.
FR-13	Manager loan assignment	Allow managers to inspect loans/employees and assign pending loan requests to active employees.
FR-14	Administrator employee/user management	Allow administrators to list users, add employees, modify user details, and manage roles.
FR-15	Password change	Allow all logged-in roles to change their password.
FR-16	Logout	Clear the login/session state for the current user.
FR-17	Server-side authorization	Validate stored active state, stored role, logged-in state, command role, and loan assignment on the server. Do not trust client role alone.
FR-18	Audit logging	Write timestamp, process ID, user ID, action, and success/failure result to stdout and bank.log.
FR-19	Persistent storage	Store records in fixed-width text files and synchronize writes with fsync().
FR-20	Concurrent handling	Serve requests concurrently using fork() and serialize access to records with fcntl record locks.

5. Non-functional requirements

ID	Requirement	Description
NFR-01	Performance	Support multiple request handlers concurrently; independent record locks should avoid unnecessarily serializing unrelated account records.
NFR-02	Correctness	Re-read records after locking; prevent negative balances; reject invalid roles, amounts, IDs, and loan assignment states.
NFR-03	Durability	Call fsync() after record writes; committed/finished operations should be checked after process/server restart.
NFR-04	Maintainability	Keep user-facing menus, server command handlers, and file record operations in separate source modules (client.c, server.c, database.c).
NFR-05	Portability	Build with GCC and standard POSIX/Linux system interfaces; no third-party libraries are listed as required.
NFR-06	Security limitation	Current implementation uses plaintext passwords and an unencrypted TCP protocol; restrict to controlled demonstration environments.
NFR-07	Auditability	Record request outcome and timestamp in bank.log.
NFR-08	Capacity	Fixed-width records bound per-field sizes; linear lookup and fixed record sizes make this a small-demo design rather than an enterprise store.

6. External interfaces

- User interface: terminal menus in client.c.
- Network: IPv4 TCP server on port 8080; one command is sent and one response returned per connection.
- Storage: five plain-text files in ./data/; bank.log is a separate append-only request outcome log.
- Build: Makefile compiles server, client, and stress_test with gcc; stress_test also links libm.

7. Constraints and assumptions

- Run executables from the project root because the data and log paths are relative.
- The protocol uses “|” separators; delimiter characters must be excluded/escaped by the caller, but no escaping scheme is specified.
- The server’s single recv() call assumes each small request arrives in the read buffer as one logical message. TCP does not guarantee that message boundaries match recv() boundaries; robust framing is a known enhancement.
- Passwords are 4–30 characters according to server validation and may not contain “|”; usernames are 3–20 alphanumeric/underscore characters according to server validation.
- Amounts use double and are rounded to two decimal places in the file layer.
- No separate account status column exists in accounts.txt; manager activation is stored in users.active.

8. Acceptance criteria

- All four roles can authenticate using valid seeded users; invalid credentials and mismatched roles fail.

- A user cannot maintain two simultaneous sessions.
- Deposits/withdrawals respect positive-amount and available-balance checks.
- Transfers preserve total balances under concurrent requests and reject self-transfer or inactive recipients.
- Only the assigned employee can process an assigned loan, and processing is not repeated.
- Manager and administrator commands are rejected when invoked by a role not permitted by the command table.
- The prescribed stress test is run on a clean data set and produces recorded actual results.